

3FS文件系统详解

3FS (Fire-Flyer 文件系统) 是 [DeepSeek AI](#) 开发的一种高性能分布式并行文件系统，专为满足 AI 训练和推理工作负载的密集数据需求而设计。它利用现代 SSD 和 RDMA 网络，提供了一个共享存储层，简化了分布式应用程序的开发，并支持 DeepSeek 的大型语言模型（如 DeepSeek-V3 和 DeepSeek-R1）的训练和推理过程。

背景与上下文

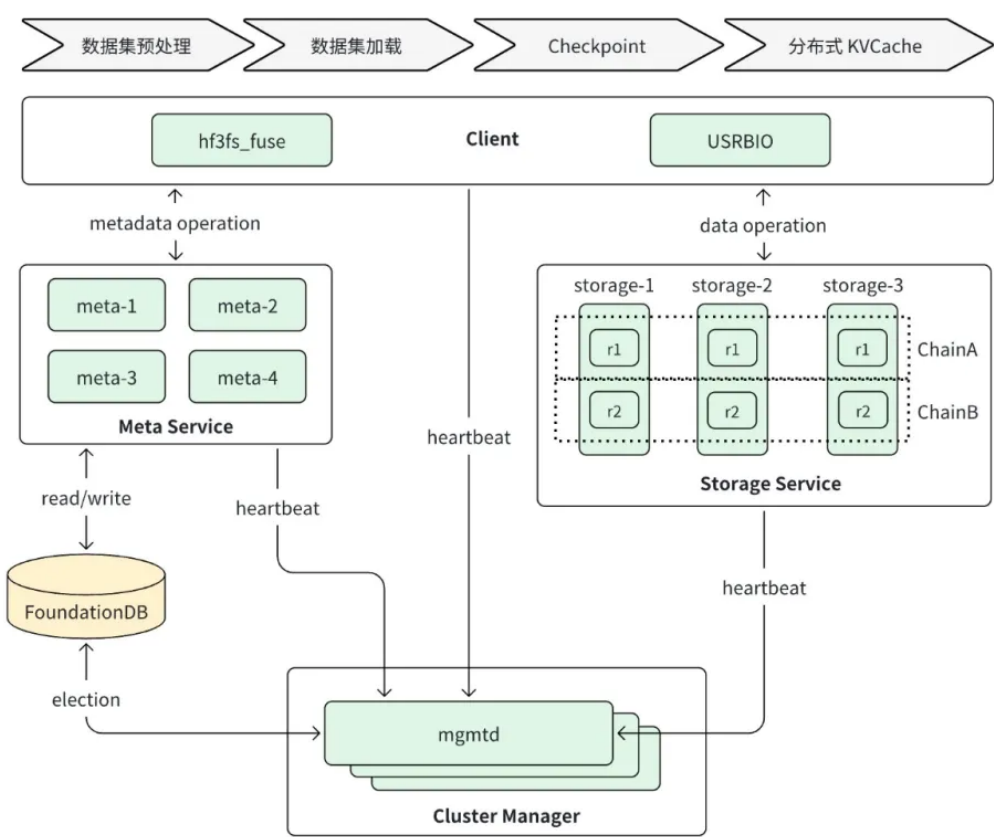
DeepSeek AI 是一家专注于开发开源大型语言模型（LLM）的中国人工智能公司，其计算基础设施包括至少两个计算集群：Fire-Flyer 和 Fire-Flyer 2。3FS 是 [Fire-Flyer 集群](#) 软件侧的关键组件，旨在处理大规模数据访问和存储需求。2025 年 2 月 27 日的当前信息显示，3FS 的开发和性能数据主要通过其 GitHub 存储库和相关文档公开。

关键特点与架构

3FS 的设计体现了以下几个核心特点：

- 分离式架构：存储和计算资源分离，支持独立扩展。这种设计允许在不影响计算节点的情况下扩展存储容量，特别适合 AI 工作负载的动态需求。
- 强一致性机制：通过 Chain Replication with Apportioned Queries ([CRAQ](#)) 确保数据在所有节点的一致性，这对于分布式系统中的数据可靠性至关重要。
- 高性能优化：3FS 使用 [Direct I/O](#) 和 [RDMA Read](#)，特别针对异步随机读取优化。Direct I/O 绕过操作系统缓冲区缓存，适合数据不重复使用的场景（如 AI 训练中的随机样本访问），而 RDMA 提供高速度网络通信，减少 CPU 介入，提高吞吐量。
- 多功能支持：3FS 支持多种 AI 工作负载，包括：
 - 数据准备：将数据分析管道的输出组织成层次目录结构，高效管理大量中间输出。
 - 训练数据加载：支持计算节点对训练样本的随机访问，消除预取或打乱数据的需要。
 - 检查点保存/重载：提供高吞吐量的并行检查点功能，适合大规模训练。
 - KVCache 用于推理：作为 DRAM 缓存的成本效益替代方案，提供高吞吐量和更大的容量，特别适用于推理任务。

3FS 整体架构



与业界很多[分布式文件系统](#)架构类似，3FS 整个系统由四个部分组成，分别是 Cluster Manager、Client、Meta Service、Storage Service。所有组件均接入 RDMA 网络实现高速互联，DeepSeek 内部实际使用的是 InfiniBand。

Cluster Manager 是整个集群的中控，承担节点管理的职责：

- Cluster Manager 采用多节点热备的方式解决自身的高可用问题，选主机制复用 Meta Service 依赖的 FoundationDB 实现；
- Meta Service 和 Storage Service 的所有节点，均通过周期性心跳机制维持在线状态，一旦这些节点状态有变化，由 Cluster Manager 负责通知到整个集群；
- Client 同样通过心跳向 Cluster Manager 汇报在线状态，如果失联，由 Cluster Manager 帮助回收该 Client 上的文件写打开状态。

Client 提供两种客户端接入方案：

- FUSE 客户端 hf3fs_fuse 方便易用，提供了对常见 POSIX 接口的支持，可快速对接各类应用，但性能不是最优的；
- 原生客户端 USBIO 提供的是 SDK 接入方式，应用需要改造代码才能使用，但性能相比 FUSE 客户端可提升 3-5 倍。

Meta Service 提供元[数据服务](#)，采用存算分离设计：

- 元数据持久化存储到 FoundationDB 中，FoundationDB 同时提供事务机制支撑上层实现文件系统目录树语义；
- Meta Service 的节点本身是无状态、可横向扩展的，负责将 POSIX 定义的目录树操作翻译成 FoundationDB 的读写事务来执行。

Storage Service 提供数据存储服务，采用存算一体设计：

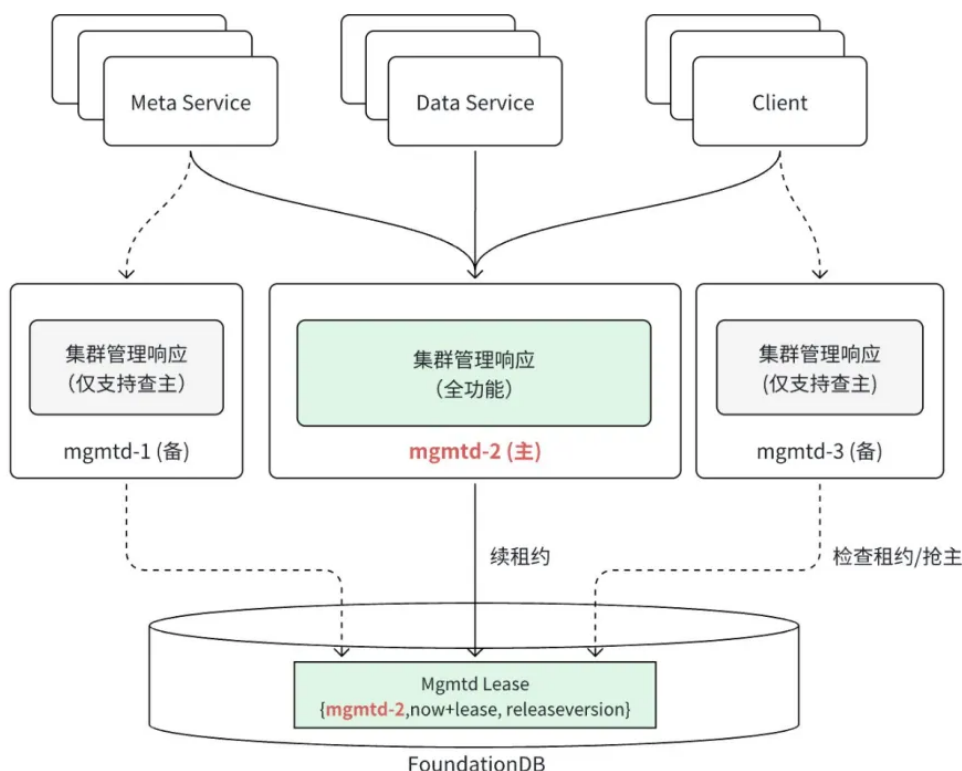
- 每个存储节点管理本地 SSD 存储资源，提供读写能力；
- 每份数据 3 副本存储，采用的链式复制协议 CRAQ（Chain Replication with Apportioned Queries）提供 write-all-read-any 语义，对读更友好；
- 系统将数据进行分块，尽可能打散到多个节点的 SSD 上进行数据和负载均摊。

3FS 架构详解

集群管理

整体架构

一个 3FS 集群可以部署单个或多个管理服务节点 mgmtd。这些 mgmtd 中只有一个主节点，承接所有的集群管理响应诉求，其它均为备节点仅对外提供查询主的响应。其它角色节点都需要定期向主 mgmtd 汇报心跳保持在线状态才能提供服务。



节点管理

每个节点启动后，需要向主 mgmtd 上报必要的信息建立租约。mgmtd 将收到的节点信息持久化到 FoundationDB 中，以保证切主后这些信息不会丢失。节点信息包括节点 ID、主机名、服务地址、节点类别、节点状态、最后心跳的时间戳、配置信息、标签、软件版本等。

租约建立之后，节点需要向主 mgmtd 周期发送心跳对租约进行续租。租约双方根据以下规则判断租约是否有效：

- 如果节点超过 T 秒（可配置，默认 60s）没有上报心跳，主 mgmtd 判断节点租约失效；
- 如果节点与主 mgmtd 超过 T/2 秒未能续上租约，本节点自动退出。

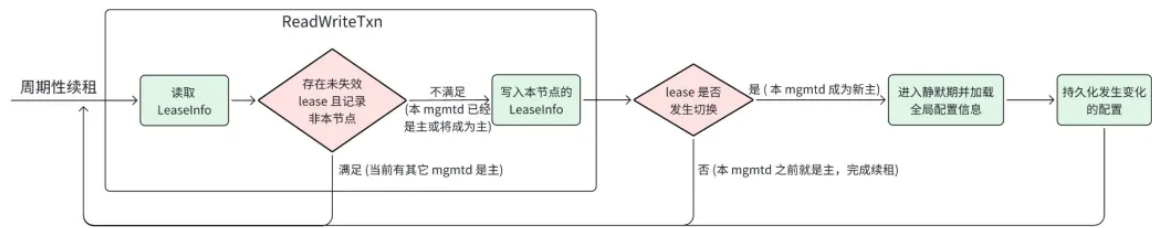
对于元数据节点和客户端，租约有效意味着服务是可用的。但对于存储服务节点，情况要复杂一些。一个存储节点上会有多个 CRAQ 的 Target，每个 Target 是否可服务的状态是不一致的，节点可服务不能代表一个 Target 可服务。因此，Target 的服务状态会进一步细分为以下几种：

状态	可读	可写	说明
serving	是	是	服务在线，可以响应客户端请求
syncing	否	是	服务在线，正在进行数据恢复
waiting	否	否	服务在线，数据恢复尚未启动
lastsrv	否	否	服务下线，且是 CRAQ 最后一个可以提供服务的 Target
offline	否	否	服务下线或磁盘介质损坏

元数据和存储节点（包括其上的 Target）的信息，以及下文会描述的 CRAQ 复制链表信息，共同组成了集群的路由信息（RoutingInfo）。路由信息由主 mgmtd 广播到所有的节点，每个节点在需要的时候通过它找到其它节点。

选主机制

mgmtd 的选主机制基于租约和 FoundationDB 读写事务实现。租约信息 LeaseInfo 记录在 FoundationDB 中，包括节点 ID、租约失效时间、软件版本信息。如果租约有效，节点 ID 记录的节点即是当前的主。每个 mgmtd 每 10s 执行一次 FoundationDB 读写事务进行租约检查，具体流程如下图所示。

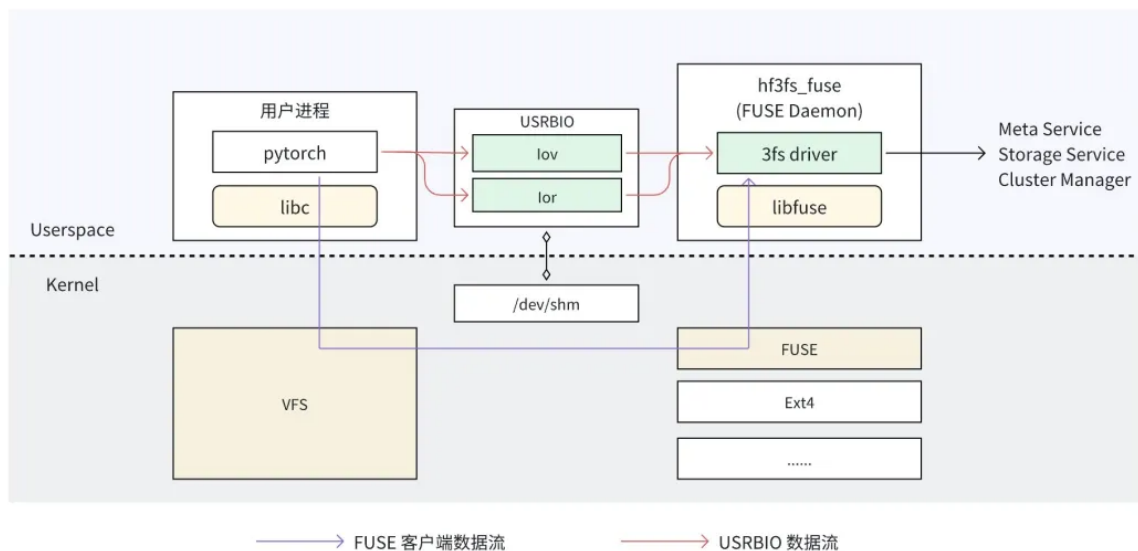


上述流程通过以下几点保证了选主机制的正确性：

- LeaseInfo 的读取和写入在同一个 FoundationDB 读写事务里完成，FoundationDB 读写事务确保了即使多个 mgmtd 并发进行租约检查，执行过程也是串行一个一个的，避免了多个 mgmtd 交织处理分别认为自己成为主的情况。
- 发生切主之后新主会静默 120s 才提供服务，远大于租约有效时长 60s，这个时间差可以保证老主上的在飞任务有充足的时间处理完，避免出现新主、老主并发处理的情况。

客户端

整体架构



3FS 提供了两种形态的客户端，FUSE 客户端 hf3fs_fuse 和原生客户端 USRBIO：

- FUSE 客户端适配门槛较低，开箱即用。在 FUSE 客户端中，用户进程每个请求都需要经过内核 VFS、FUSE 转发给用户态 FUSE Daemon 进行处理，存在 4 次“内核 - 用户态”上下文切换，数据经过 1-2 次拷贝。这些上下文切换和数据拷贝开销导致 FUSE 客户端的性能存在瓶颈；
- USRBIO 是一套用户态、异步、零拷贝 API，使用时需要业务修改源代码来适配，使用门槛高。每个读写请求直接从用户进程发送给 FUSE Daemon，消除了上下文切换和数据拷贝开销，从而实现了极致的性能。

FUSE 客户端

FUSE 客户端基于 libfuse lowlevel api 实现，要求 libfuse 3.16.1 及以上版本。和其它业界实现相比，最大的特色是使用了 C++20 协程，其它方面大同小异。本文仅列举一些实现上值得注意的点：

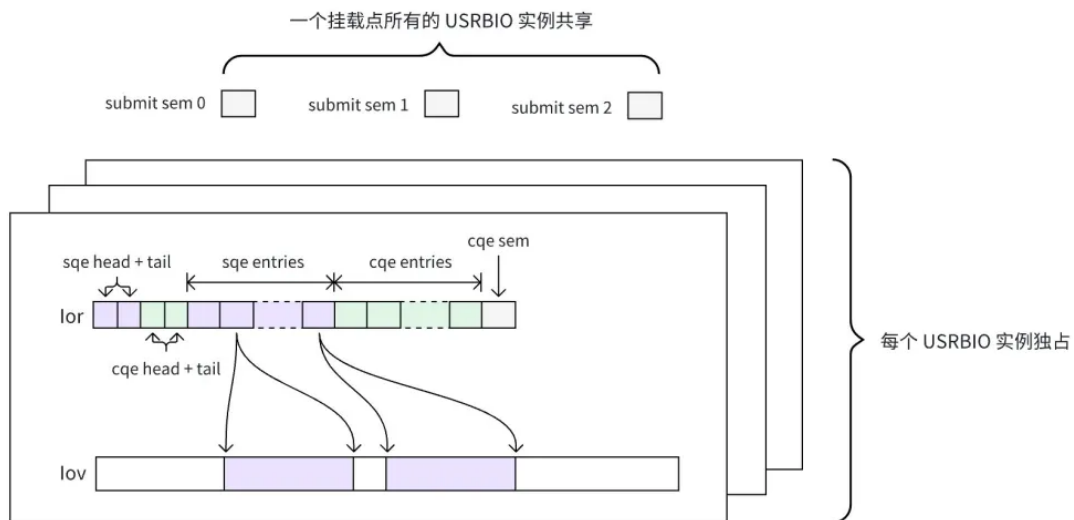
分类	实现	分析
POSIX 兼容性	不支持文件锁、xattr，其它主流操作均支持。	文件锁功能对于有多机强一致性要求的业务（例如数据库）比较重要。
目录遍历	实现了 readdirplus 版本的接口。 一次完整的目录遍历会包含一次 opendir、一次 closedir，和多次 readdirplus 调用。 第一次调用 readdirplus 时，3FS 分批从 Meta Service 获取到完整的文件列表（不包含文件属性），缓存到内存中。这个缓存会在 closedir 的时候释放。 每次 readdirplus 时，从缓存中拿到本次要返回的文件列表，向 Meta Service 发送一次 RPC 批量获取到这些文件的属性。	这个实现对文件属性的获取有较多优化： 1. readdirplus 版本相比 readdir 版本会将文件属性信息一并带回给内核缓存，可以有效减少接下来一段时间的 stat 调用 2. 在获取属性时采取了 Batch 方式，比并行发多个操作更高效 这个实现对于有上千万甚至上亿文件的巨型目录不够友好： 1. 第一次 readdirplus 调用会卡住非常长的时间 2. 缓存住全部的文件列表需要的内存较多 这些缺点均可以在业务侧合理控制目录大小来规避。
预读	FUSE readahead 参数默认调整到 16MB。	16MB readahead 参数对顺序读场景（例如模型加载）比较友好。
读写缓存	dio 写直接落后端。 buffered io 启用了 page cache。每个文件的写只有单路并发，会维护一个 1MB（可调）write buffer，顺序写积攒一下再下刷。	write buffer 的设计是一个面向写吞吐优化的设计，对顺序写比较友好。
文件长度延迟更新	数据下刷后不会马上向 Meta Service 更新文件 size，而是由后台任务周期（默认 30s）或者 close、fsync 刷新。	文件长度延迟更新对降低 Meta Service 的压力有帮助，但是放松了写数据的一致性。
异步 close	close 操作会立刻返回，实际操作后台执行。	这种做法可以将 close 的延时隐藏掉。
写打开文件延迟删除	文件以写模式打开时，会和 Meta Service 会建立 session，可以保护文件在使用过程中不被删除掉。 文件以读模式打开时没有此保护，在读的过程中文件如果被删除会报错。	分开处理读写模式在语义和效率之间取得一定平衡。
递归删除目录	提供了定制的 rm -rf 的能力，将目录直接发到 Meta Service 后台处理，客户端直接返回。	rm -rf 在现实中是非常高频的使用方式，但是大目录执行会非常慢，这是因为 POSIX 标准并没有定义递归删除目录的接口，会以“递归遍历所有目录 + 单并发删除”的方式模拟。 3FS 的实现方式借鉴了 HDFS 的实现，无论是实现效率还是用户体验都会有大幅提升。

USBIO

基于共享内存 RingBuffer 的通信机制被广泛应用在[高性能存储](#)、网络领域，在 DPDK、io_uring 中均有相关实现，一般采用无锁、零拷贝设计，相比其它通信的机制有明显的性能提升。3FS 借鉴了这个思路实现了 USBIO，和原有的 FUSE 实现相比，有以下特点：

- 整个执行路径非常精简，完全在用户态实现，不再需要陷入内核经过 VFS、FUSE 内核模块的处理
- 读写数据的 buffer 和 RDMA 打通，整个处理过程没有拷贝开销
- 只加速最关键的读写操作，其它操作复用 FUSE 现有逻辑，在效率和兼容性之间取得了极佳的平衡。这一点和 GPU Direct Storage 的设计思路有异曲同工之处

USBIO 的使用说明可以参考 3FS 代码库 USBIO API Reference 文档：<https://github.com/deepseek-ai/3FS/blob/main/src/lib/api/Usrblo.md>



在实现上，USBIO 使用了很多共享内存文件：

- 每个 USBIO 实例使用一个 lov 文件和一个 lor 文件
 - lov 文件用来作为读写数据的 buffer
 - 用户提前规划好需要使用的总容量
 - 文件创建之后 FUSE Daemon 将该其注册成 RDMA memory buffer，进而实现整个链路的零拷贝
- lor 文件用来实现 loRing
 - 用户提前规划好并发度
 - 在整个文件上抽象出了提交队列和完成队列，具体布局参考上图
 - 文件的尾部是提交完成队列的信号量，FUSE Daemon 在处理完 IO 后通过这个信号量通知到用户进程
- 一个挂载点的所有 USBIO 共享 3 个 submit sem 文件
 - 这三个文件作为 IO 提交事件的信号量（submit sem），每一个文件代表一个优先级
 - 一旦某个 USBIO 实例有 IO 需要提交，会通过该信号量通知到 FUSE Daemon
- 所有的共享内存文件在挂载点 3fs-virt/iops/ 目录下均建有 symlink，指向 /dev/shm 下的对应文件

lov、lor 共享内存文件通过 symlink 注册给 FUSE Daemon，这也是 3FS FUSE 实现上有意思的一个点，下一章节还会有进一步的描述。

symlink 黑魔法

通常一个文件系统如果想实现一些非标能力，在 `ioctl` 接口上集成是一个相对标准的做法。3FS 里除了使用了这种方式外，对于 `USBIO`、递归删除目录、禁用回收站的 `rename`、修改 `conf` 等功能，采用了集成到 `symlink` 接口的非常规做法。

3FS 采用这种做法可能基于两个原因：

- `ioctl` 需要提供专门的工具或写代码来使用，但 `symlink` 只要有挂载点就可以直接用。
- 和其它接口相比，`symlink` 相对低频、可传递的参数更多。

`symlink` 的完整处理逻辑如下：

- 当目标目录为挂载点 `3fs-virt` 下的 `rm-rf`、`iovs`、`set-conf` 目录时：
 - `rm-rf`：将 `link` 路径递归删除，请求发送给元数据服务处理；
 - `iovs`：建 `lov` 或者 `lor`，根据 `target` 文件后缀判定是否 `ior`；
 - `set-conf`：设置 `config` 为 `target` 文件中的配置。
- 当 `link` 路径以 `mv`：开头，`rename` 紧跟其后的 `link` 文件路径到 `target` 路径，禁用回收站。
- 其它 `symlink` 请求 `Meta Service` 进行处理。

FFRecord

3FS 没有对小文件做调优，直接存取大量小文件性能会比较差。为了弥补这个短板，3FS 专门设计了 `FFRecord`（Fire Flyer Record）文件格式来充分发挥系统的大 IO 读写能力。

`FFRecord` 文件格式具有以下特点：

- 合并多个小文件，减少了训练时打开大量小文件的开销；
- 支持随机批量读取，提升读取速度；
- 包含数据校验，保证读取的数据完整可靠。

以下是 `FFRecord` 文件格式的存储 layout:

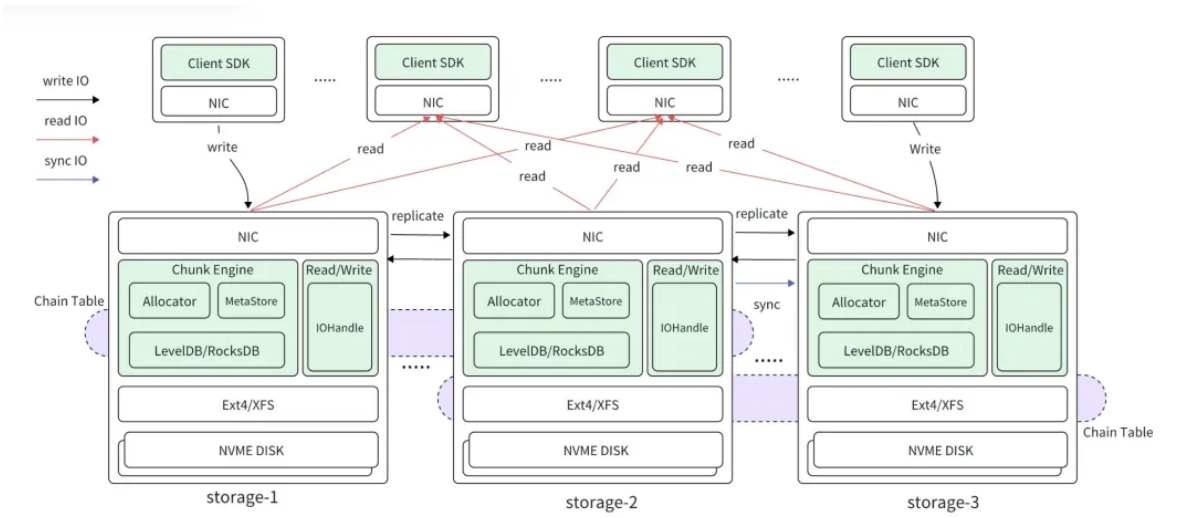
checksum		N	
checksums		offsets	
sample1	sample2		sample N

图片在 `FFRecord` 文件格式中，每一条样本的数据会做序列化按顺序写入，同时文件头部包含了每一条样本在文件中的偏移量和 `crc32` 校验和，方便做随机读取和数据校验。

存储服务

整体架构

3FS 面向高吞吐能力而设计，系统吞吐能力跟随 SSD 和网络带宽线性扩展，即使发生个别 SSD 介质故障，也能依然提供很高的吞吐能力。3FS 采用分摊查询的链式复制 CRAQ 来保证数据可靠性，CRAQ 的 write-all-read-any 特性对重读场景非常友好。



每个数据节点通过 Ext4 或者 XFS 文件系统管理其上的多块 NVME DISK，对内部模块提供标准的 POSIX 文件接口。数据节点包含几个关键模块：Chunk Engine 提供 chunk 分配管理；MetaStore 负责记录分配管理信息，并持久化到 RocksDB 中；主 IO handle 提供正常的读写操作。各个数据节点间组成不同的链式复制组，节点之间有复制链间写 IO、数据恢复 sync 写 IO。

CRAQ

链式复制是将多个数据节点组成一条链 chain，写从链首开始，传播到链尾，链尾写完后，逐级向前发送确认信息。标准 CRAQ 的读全部由链尾处理，因为尾部才是完全写完的数据。

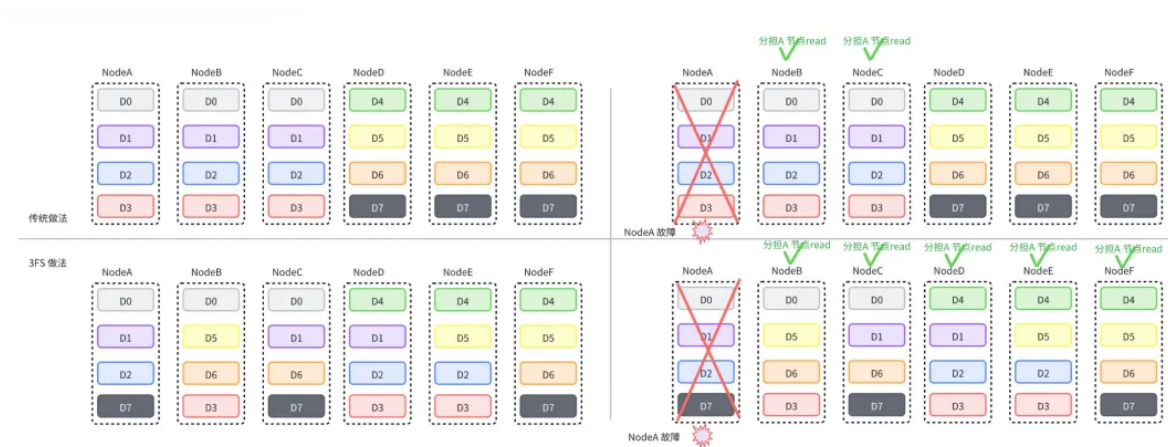
多条链组成 chain table，存放在元数据节点，Client 和数据节点通过心跳，从元数据节点获取 chain table 并缓存。一个集群可有多条 chain table，用于隔离故障域，以及隔离不同类型（例如离线或在线）的任务。

3FS 的写采用全链路 RDMA，链的后继节点采用单边 RDMA 从前序节点读取数据，相比前序节点通过 RDMA 发送数据，少了数据切包等操作，性能更高。而 3FS 的读，可以向多个数据节点同时发送读请求，数据节点通过比较 **commit version** 和 **update version** 来读取已经提交数据，多节点的读相比标准 CRAQ 的尾节点读，显著提高吞吐。

数据打散

传统的链式复制以固定的节点形成 chain table。如图所示节点 NodeA 只与 NodeB、C 节点形成 chain。若 NodeA 故障，只能 NodeB 和 C 分担读压力。

3FS 采用了分摊式的打散方法，一个 Node 承担多个 chain，多个 chain 的数据在集群内多个节点进行数据均摊。如图所示，节点 NodeA 可与 Node B-F 节点组成多个 chain。若 NodeA 产生故障，NodeB-F 更多节点分担读压力，从而可以避免 NodeA 节点故障的情况下，产生节点读瓶颈。



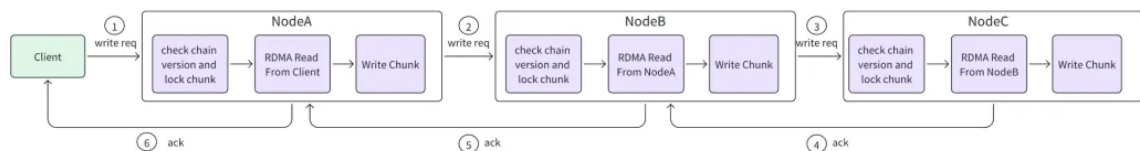
文件创建流程

- 步骤 1：分配 FoundationDB 读写事务；
- 步骤 2：事务内写目标文件的 dentry、inode；创建文件是继承父目录 layout，根据 stripe size 选取多条 chain，并记录在 inode 中；写打开创建场景，还会写入对应 file session；
- 步骤 3：事务内将父目录 inode、目标 dentry 加入读冲突列表。保证父目录未被删除，及检查目标文件已存在场景；
- 步骤 4：提交读写事务。

读写流程

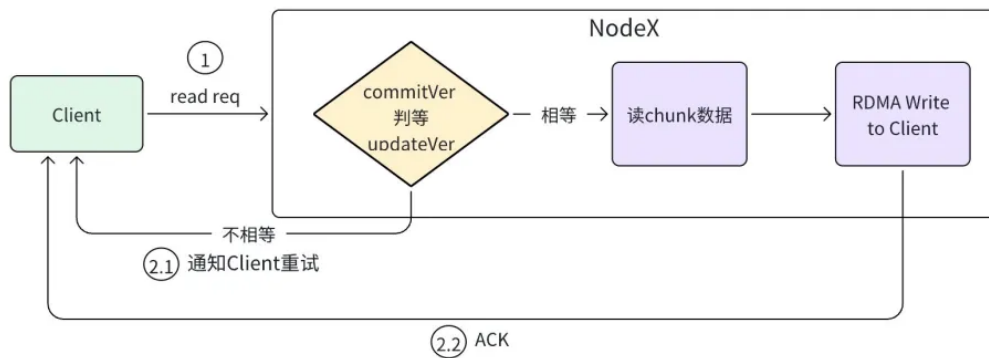
写数据流程：

- 步骤 1：Client 获取数据的目标 chain，并向 chain 首节点 NodeA 发送写请求；
- 步骤 2：NodeA 检查 chain version 并锁住 chunk，保证对同一 chunk 的串行写，再用单边 RDMA 从 client 读取数据，写入本地 chunk，记录 updateVer；
- 步骤 3：NodeA 将写请求传播到 NodeB 和 NodeC，NodeB 和 NodeC 处理逻辑和 NodeA 相同；
- 步骤 4：chain 尾节点 NodeC 写完数据后，将回复传播到 NodeB，NodeB 更新 commitVer 为 updateVer；
- 步骤 5：NodeB 将回复传播到 NodeA，NodeA 处理同 NodeB；
- 步骤 6：NodeA 回复 Client 写完成。



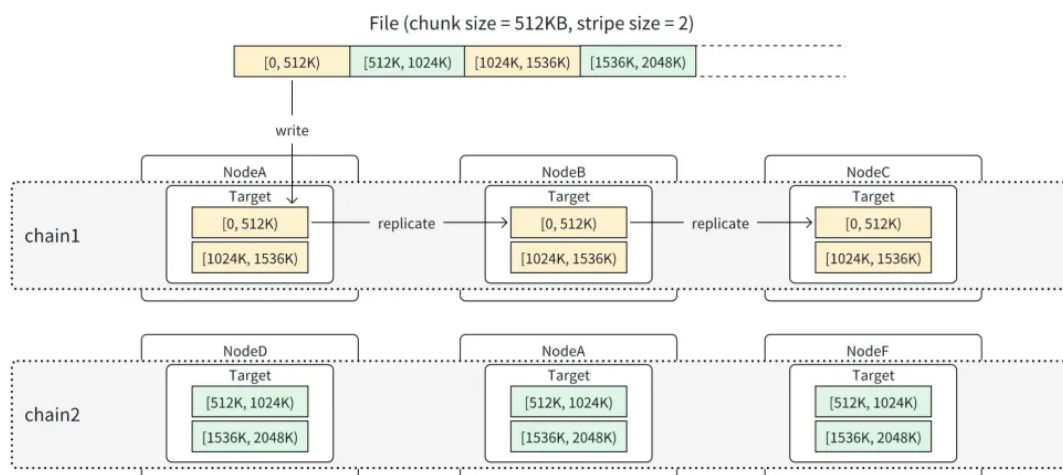
读数据流程：

- 步骤 1：Client 获取数据所在的 chain，并向 chain 某个节点 NodeX 发送读请求；
- 步骤 2：NodeX 检查本地 commitVer 和 updateVer 是否相等；
 - 步骤 2.1：如果不等，说明有其它 flying 的写请求，通知 Client 重试；
 - 步骤 2.2：如果相等，则从本地 chunk 读取数据，并通过 RDMA 写给 Client；



文件布局

一个文件在创建时，会按照父目录配置的 layout 规则，包括 chain table 以及 stripe size，从对应的 chain table 中选择多个 chain 来存储和并行写入文件数据。chain range 的信息会记录到 inode 元数据中，包括起始 chain id 以及 seed 信息（用来做随机打散）等。在这个基础之上，文件数据被进一步按照父目录 layout 中配置的 chunk size 均分成固定大小的 chunk（官方推荐 64KB、512KB、4MB 3 个设置，默认 512KB），每个 chunk 根据 index 被分配到文件的一个 chain 上，chunk id 由 inode id + track + chunk index 构成。当前 track 始终为 0，猜测是预留给未来实现 chain 动态扩展用的。



访问数据时用户只需要访问 Meta Service 一次获得 chain 信息和文件长度，之后根据读写的字节范围就可以计算出由哪些 chain 进行处理。

假设一个文件的 chunk size 是 512KB，stripe size 是 200，对应的会从 chain table 里分配 200 个 chain 用来存储这个文件的所有 chunk。在文件写满 100MB (512KB * 200) 之前，其实并不是所有的 chain 都会有 chunk 存储。在一些需要和 Storage Service 交互的操作中，比如计算文件长度（需要获得所有 chain 上最后一个 chunk 的长度）、或者 Truncate 操作，需要向所有潜在可能存放 chunk 的 Storage Service 发起请求。但是对不满 100MB（不满 stripe size 个 chunk）的小文件来说，向 200 个 chain 的 Storage Service 都发起网络请求无疑带来无谓的延时增加。

为了优化这种场景，3FS 引入了 Dynamic Stripe Size 的机制。这个的作用就是维护了一个可能存放有 chunk 的 chain 数量，这个值类似 C++ vector 的扩容策略，每次 x2 来扩容，在达到 stripe size 之后就不再扩了。这个值的作用是针对小文件，缩小存放有这个文件数据的 chain 范围，减少需要和 Storage Service 通信的数量。

通过固定切分 chunk 的方式，能够有效的规避数据读写过程中与 Meta Service 的交互次数，降低元数据服务的压力，但是也引入另外一个弊端，即对写容错不够友好，当前写入过程中，如果一个 chunk 写失败，是不支持切下一个 chunk 继续写入的，只能在失败的 chunk 上反复重试直到成功或者超时失败。

单机引擎

Chunk Engine 由 chunk data file、Allocator、LevelDB/RocksDB 组成。其中 chunk data file 为数据文件；Allocator 负责 chunk 分配；LevelDB/RocksDB 主要记录本地元数据信息，默认使用 LevelDB。

为确保查询性能高效，内存中全量保留一份元数据，同时提供线程级安全的访问机制，API 包括：

API 接口	作用
Open/Close	初始化 Engine，从 RocksDB 中加载元数据
Get	通过 hash map cache 获取元数据引用计数
Update	采用 COW 的方式，以整块的方式进行更新，旧块可读，直到无引用才释放
Commit	批量提交的方式写入 LevelDB/RocksDB

Chunk

Chunk 大小范围 64KiB-64MiB，按照 2 的幂次递增，共 11 种，Allocator 会选择最接近实际空间大小的物理块进行分配。

对于每种物理块大小，以 256 个物理块组成一个 Resource Pool，通过 Bitmap 标识空间状态，为 0 代表空闲可回收状态，分配的时候优先分配空闲可回收的物理块。

写入流程

- 修改写：采用 COW 的方式，Allocator 优先分配新的物理块，系统读取已经存在的 Chunk Data 到内存，然后填充 update 数据，拼装完成后写入新分配的物理块；
- 尾部 Append 写：数据直接写入已存在 block，会新生成一份元数据包括新写入的 location 信息和已经存在的 chunk meta 信息，原子性写入到 LevelDB 或 RocksDB 中，以避免覆盖写带来的写放大。

数据恢复

存储服务崩溃、重启、介质故障，对应的存储 Target 不参与数据写操作，会被移动到 chain 的末尾。当服务重新启动的时候，offline 节点上对应存储 Target 的数据为老数据，需要与正常节点的数据进行补齐，才能保证数据一致性。offline 的节点周期性的从 cluster manager 拉取最新的 chain table 信息，直到该节点上所有的存储 Target 在 chain table 中都被标记为 offline 以后，才开始发送心跳。这样可以保证该节点上的所有存储 Target 各自独立进入恢复流程。数据恢复采用了一种 full-chunk-replace 写的方式，支持边写边恢复，即上游节点发现下游的 offline 节点恢复，开始通过链式复制把写请求转发给下游节点，此时，哪怕 Client 只是写了部分数据，也会直接把完整的 chunk 复制给下游，实现 chunk 数据的恢复。

数据恢复过程整体分成为两个大步骤：Fetch Remote Meta、Sync Data。其中 Local node 代表前继正常节点，Remote node 为恢复节点。

数据恢复流程

- 步骤 1：Local Node 向 Remote Node 发起 meta 获取，Remote Node 读取本地 meta；
- 步骤 2：Remote Node 向 Local Node 返回元数据信息，Local Node 比对数据差异；

- 步骤 3: 若数据有差异, Local Node 读取本地 chunk 数据到内存;
- 步骤 4: Remote Node 单边读取 Local Node chunk 内存数据;
- 步骤 5: Remote Node 申请新 chunk, 并把数据写入新 chunk。

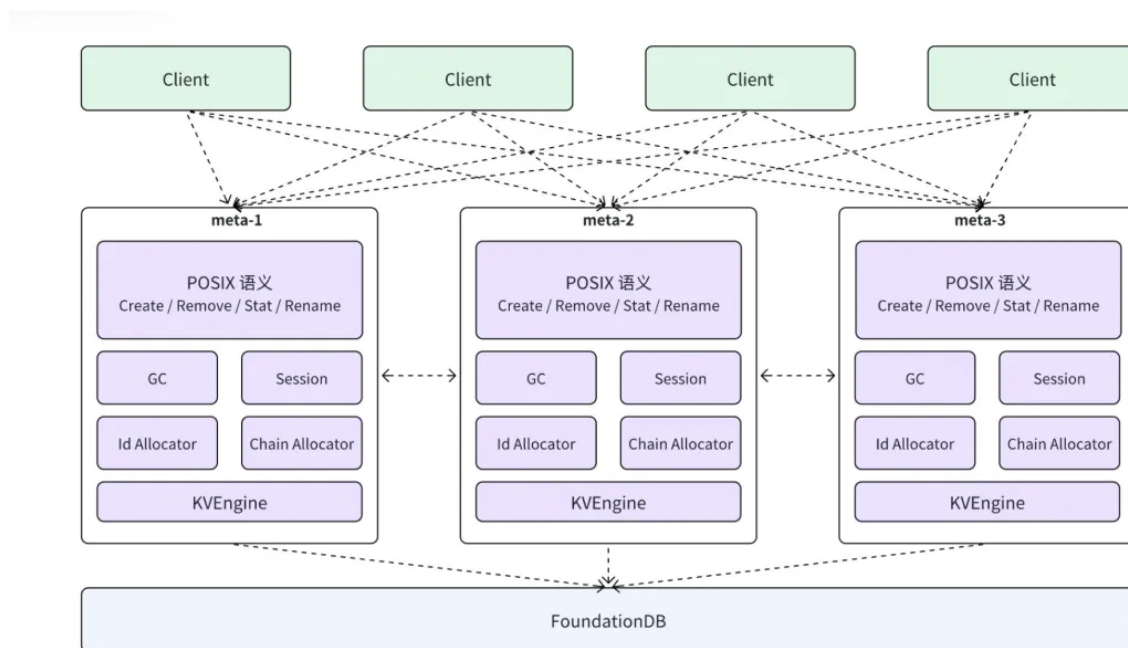
Sync Data 原则:

- 如果 chunk Local Node 存在 Remote Node 不存在, 需要同步;
- 如果 Remote Node 存在 Local Node 不存在, 需要删除;
- 如果 Local Node 的 chain version 大于 Remote Node, 需要同步;
- 如果 Local Node 和 Remote Node chain version 一样大, 但是 commit version 不同, 需要同步;
- 其他情况, 包括完全相同的数据, 或者正在写入的请求数据, 不需要同步。

元数据服务

整体架构

业界基于分布式高性能 KV 存储系统, 构建大规模文件系统元数据组件已成共识, 如 Google Colossus、Microsoft ADLS 等。3FS 元数据服务使用相同设计思路, 底层基于支持事务的分布式 KV 存储系统, 上层元数据代理负责对外提供 POSIX 语义接口。总体来说, 支持了大部分 POSIX 接口, 并提供通用元数据处理能力: inode、dentry 元数据管理, 支持按目录继承 chain 策略、后台数据 GC 等特性。



3FS 选择使用 FoundationDB 作为底层的 KV 存储系统。FoundationDB 是一个具有事务语义的分布式 KV 存储, 提供了 NoSQL 的高扩展, 高可用和灵活性, 同时保证了 serializable 的强 ACID 语义。该架构简化了元数据整体设计, 将可靠性、扩展性等分布式系统通用能力下沉到分布式 KV 存储, Meta Service 节点只是充当文件存储元数据的 Proxy, 负责语义解析。

利用 FoundationDB SSI 隔离级别的事务能力, 目录树操作串行化, 冲突处理、一致性问题等都交由 FoundationDB 解决。Meta Service 只用在事务内实现元数据操作语义到 KV 操作的转换, 降低了语义实现复杂度。

存算分离架构下, 各 Metadata Service 节点无状态, Client 请求可打到任意节点。但 Metadata Service 内部有通过 inode id hash, 保证同目录下创建、同一文件更新等请求转发到固定元数据节点上攒 Batch, 以减少事务冲突, 提升吞吐。计算、存储具备独立 scale-out 能力。

数据模型

Metadata Service 采用 inode 和 dentry 分离的设计思路，两种数据结构有不同的 schema 定义。具体实现时，采用了“将主键编码成 key，并添加不同前缀”的方式模拟出两张逻辑表，除主键外的其它的字段存放到 value 中。

Table 名	key	value	分析
Dentry Table	"DENT" + parent inode_id + entry name	parent_id, name, inode_type, dirAcl, uuid, gcInfo	Inode 和 Dentry 编码到两张逻辑表，且 FoundationDB 按 Range 分片，全局有序，从而将不同类型数据从物理上隔离开，对 Readdir 场景遍历 dentry 友好 Dentry Table 的 Key 把父目录 inode id 作为了前缀，同一目录下的 dentry 物理上是连续放置，Readdir 请求可通过 Scan 连续读取
Inode Table	"INOD" + inode_id	基本属性： user, group, permissions; access/modification/change times 附加属性： 1. File: file length, chunk size 2. Directory: parent inode_id, 子目录 / 文件默认布局配置(链表、块大小、条带大小) 3. Symlink: 目标路径	inode_id 小端存储，利用 inode id 连续分配，及 FoundationDB 全局有序特性，可以将 inode 散列到 DB 侧的不同数据分片上，点读(GetAttr)请求集群内分布更均衡，提升系统吞吐 同一文件的 inode 和 dentry 并未做数据亲和处理，dentry 和 inode 可能落在 FoundationDB 不同的数据分片上，Lookup 时需要跨分片读一次 inode

语义实现

在定义好的 inode、entry 结构之上，如何通过 FoundationDB 的读写事务正确实现各类 POSIX 元数据操作，是 Meta Service 中最重要的问题。但 POSIX 元数据操作有很多种，穷举说明会导致文章篇幅过长。本章节我们从这些操作中抽取了几种比较有代表性的常见操作来展开说明。

分类	实现	分析
文件创建	同一个读写事务内：写目标文件的 dentry、inode、file ssession(写打开创建)；将父目录inode、目标dentry 加入读冲突列表。 创建文件时，根据父目录配置的条带配置，从 chain table 中选取 replication chains。创目录时则继承自父目录的条带配置。 1. 孤儿节点 文件创建和父目录删除并发场景，可能导致孤儿节点。3FS 通过在同一读写事务中把父目录 inode 加入读冲突列表(addConflictRange)，避免父目录被删除导致孤儿节点 2. inode_id 分配 Metadata Service 以攒 Batch(一次 4096 个) 方式分配 inode_id，并通过拆 key (默认32)，进一步减少事务冲突。	1. Metadata 内部通过 parent inode_id hash 保证同目录下创建请求发到固定的元数据节点上，并在元数据节点上对这些请求进行合并处理，减小事务冲突和提高吞吐 2. 依靠 FoundationDB 的事务能力使文件创建操作变的简单，但没有元数据缓存，每次都需要先从底层读取父目录 inode 信息，延迟做不到极致 3. 标准的 POSIX 语义创建文件、目录需要更新父目录属性(nlink、atime/ctime)，3FS 为减少冲突没有更新父目录属性，这样的裁剪型做法，更偏特定场景性能极致优化，忽略通用文件语义的需求 4. 每个节点批量分配 inode_id，提升了文件创建性能
文件删除	同一个读写事务内：目标文件 nlink 数减一，并将原 dentry 删除，当 Inode 的 nlink 数减到 0 时，在 GcDirectory 下创建对应的 dentry，支持非空目录的删除 支持单独工具将目录移入 trash 目录，并设定过期设置。	删除文件或目录时属于标记删除，后续后台异步删除数据 有限支持了回收站功能，并设置 ttl 支持非空目录删除（非 POSIX 语义），在特定场景下比较有优势，用户不需要将文件——读取再删除，可以快速删除一个大目录
List 目录	一个只读事务内：利用 FoundationDB 的快速读，根据 parent id 前缀扫描获取 dentry 列表	设计上利用了 FoundationDB 全局有序特性，并对同一目录下dentry 的 key 加上了父目录 inode_id，巧妙的将 Readdir 请求转化为对 KV 侧的 Scan
文件 Rename	通过 FoundationDB 的读写事务一次性完成 Rename 操作，Rename 操作需要解决如下问题： 1. 孤儿节点 Rename 的目标父目录在 Rename 过程中正在被其它的节点并发删除，导致孤儿节点，解决方案与文件创建流程类似。 2. 并发成环问题 并发操作导致路径成环，通过单个 SSI 读写事务递归读取目标路径至根路径的所有 entry，一旦有并发 Rename，可以检测到冲突，从而防止成环	3FS 通过 FoundationDB 的读写事务能力解决了孤儿节点、成环等问题，保证了操作的原子性 在任意节点都可以进行 Rename，利用读写事务来完成加锁的能力，不需要将请求发送至特定的 Rename 服务进行加锁，解决了 Rename 单瓶颈点问题，扩展性更好 不支持父目录 mtime 等信息的修改，非完整兼容 posix 语义 需要目标路径递归至根目录，目录较深场景会导致 Rename 延迟偏高
文件写（Open、Close）	写 Open、Close 时会持久化 session，记录文件写打开状态。后台 GC 遇已删除，但仍 Open 着的文件不做数据清理，至文件完全 Close。	解决了 Open、Unlink 并发场景可能产生垃圾文件的问题 持久化 session，会将写 Open 会转为一次元数据写操作，增加了 Open 延时
文件 Size 更新	一个读写事务内：更新文件 inode，并把文件 inode 加入冲突列表，避免其它节点并发修改了 inode。 size 更新逻辑为：客户端上报最大写入位置，若超过了 inode 中存储的长度，且没有并发的截断操作，则该位置会被采纳为新的文件长度。 1. 针对客户端挂掉未上报最新 size 的场景，通过 query 所有的 chunks 获取最后一个 chunk 的 id 和长度	Meta Service 内部通过 inode_id hash 保证同一文件 size 的更新转发到特定元数据节点上，并在元数据节点上对这些请求进行合并处理，减小并发写入场景 FoundationDB 事务冲突导致的性能下降

结 语

本文带着读者深入到了 3FS 系统内部去了解其各个组成部分的关键设计。在这个过程中，我们可以看到 3FS 的很多设计都经过了深思熟虑，不可否认这是一个设计优秀的作品。但是，我们也注意到这些设计和目前文件存储领域的一些主流做法存在差异。

本文是系列文章的上篇，在下篇文章中我们将进一步将 3FS 和业界的一些知名的文件系统进行对比，希望能够从整个文件存储领域的角度为读者分析清楚 3FS 的优点和局限性，并总结出我们从 3FS 得到的启示，以及我们是如何看待这些启示的。